Introduction to Graphs

MODULE V PART 2

Breadth First Search (BFS) Algorithm

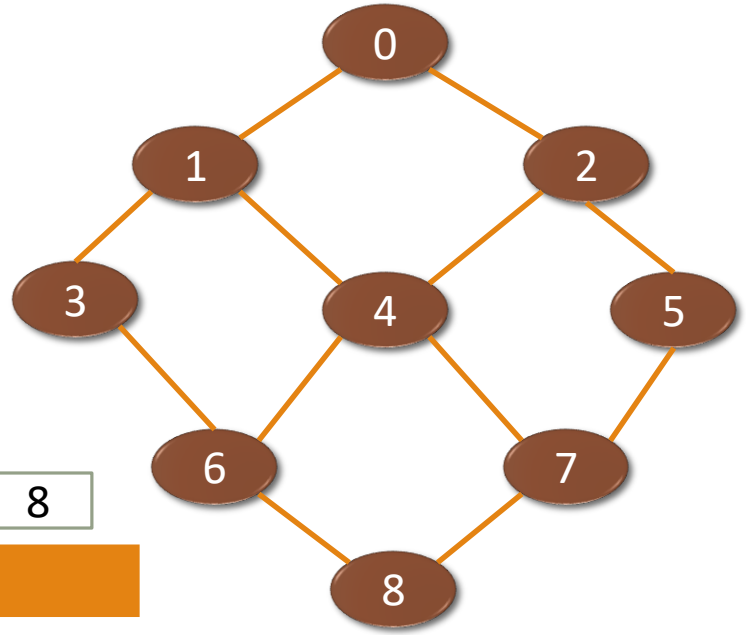
1. Enqueue starting index.
2. while Q is not empty

   Do

      P=Dequeue( )

      Enqueue all adjacent vertices of P. (Check visited Array)

   Done



Queue

0	1	2	3	4	5	6	7	8

Visited array

0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0

Breadth First Search (BFS) Algorithm

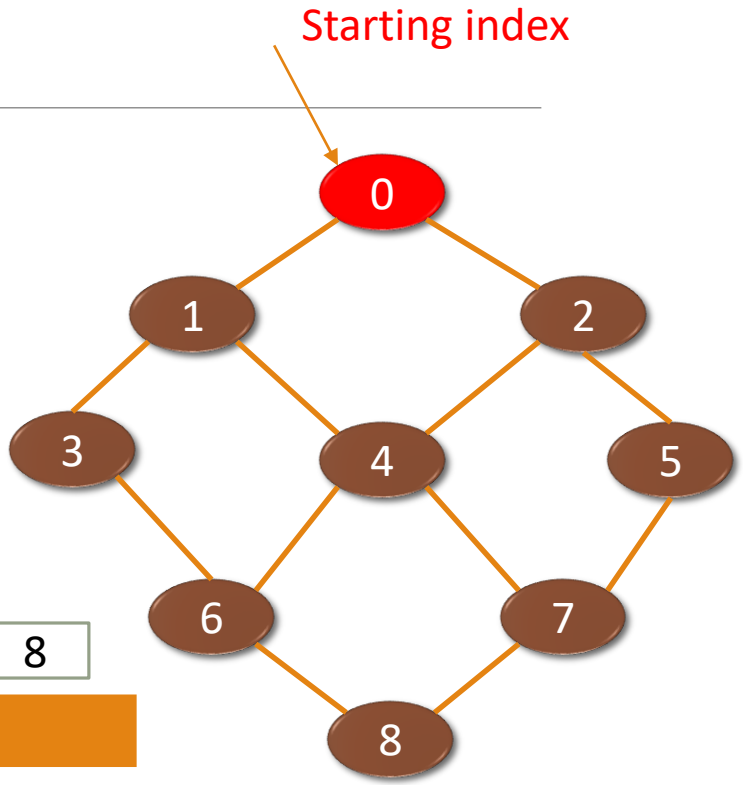
1. Enqueue starting Index and print
2. while Q is not empty

   Do

      P=Dequeue( )

      Enqueue all adjacent vertices of P. (Check visited Array)

   Done



Queue

0	1	2	3	4	5	6	7	8
0								

Visited array

0	1	2	3	4	5	6	7	8
1	0	0	0	0	0	0	0	0

Output

0

Breadth First Search (BFS) Algorithm

1. Enqueue starting index.
2. while Q is not empty

   Do

   P=Dequeue( )

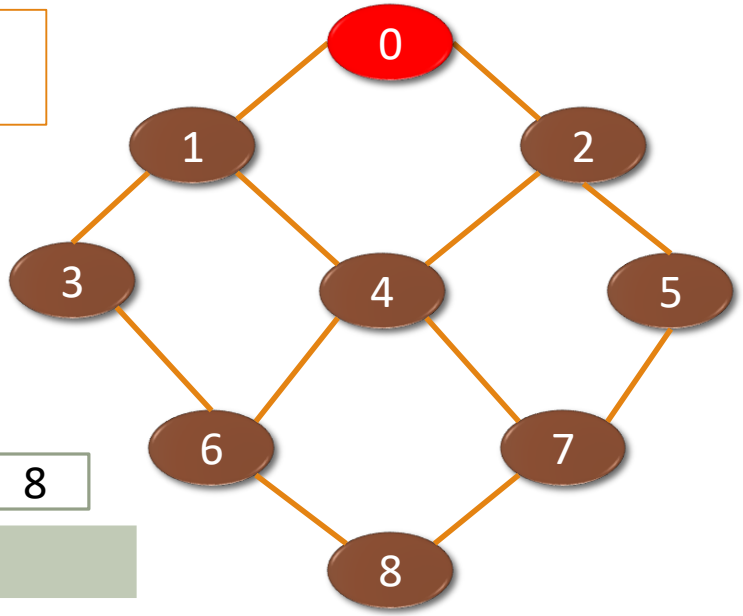
   Enqueue all adjacent vertices of P.

   (Check visited Array)

   Done

P= {0}

Starting index



Queue

0	1	2	3	4	5	6	7	8
~~0~~								

Visited array

0	1	2	3	4	5	6	7	8
1	0	0	0	0	0	0	0	0

Output

0

Breadth First Search (BFS) Algorithm

1. Enqueue starting index.
2. while Q is not empty

   Do

      P=Dequeue( )

      Enqueue all adjacent vertices of P and print

      (Check visited Array)

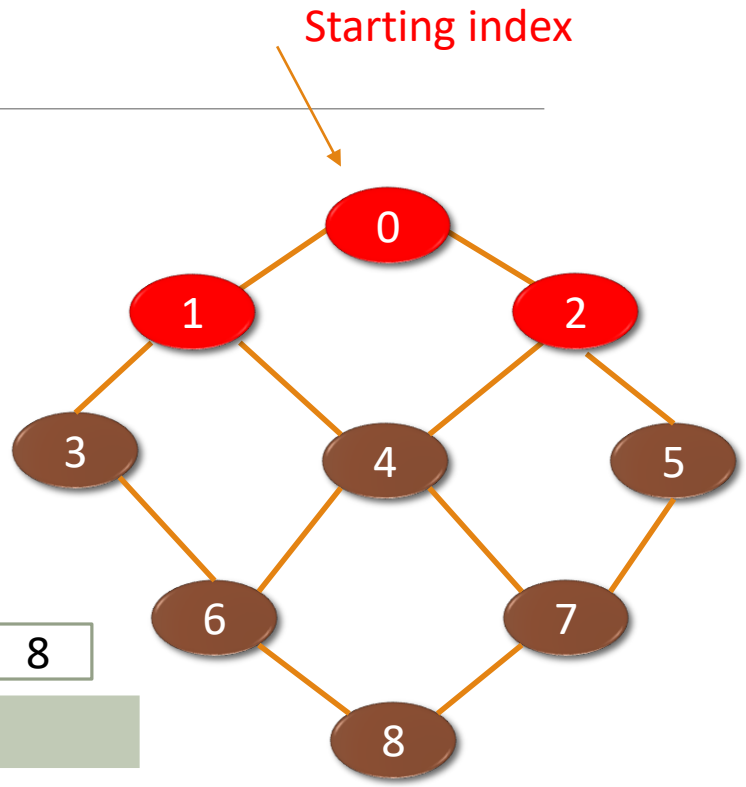
   Done

P= {0}

Adj[0] ={ 1, 2}

Enqueue(1)

Enqueue(2)



Queue

0	1	2	3	4	5	6	7	8
~~0~~	1	2						

Visited array

0	1	2	3	4	5	6	7	8
1	1	1	0	0	0	0	0	0

Output

0 1 2

Breadth First Search (BFS) Algorithm

1. Enqueue starting index.
2. while Q is not empty

   Do

   P=Dequeue( )

   Enqueue all adjacent vertices of P and print

   (Check visited Array)

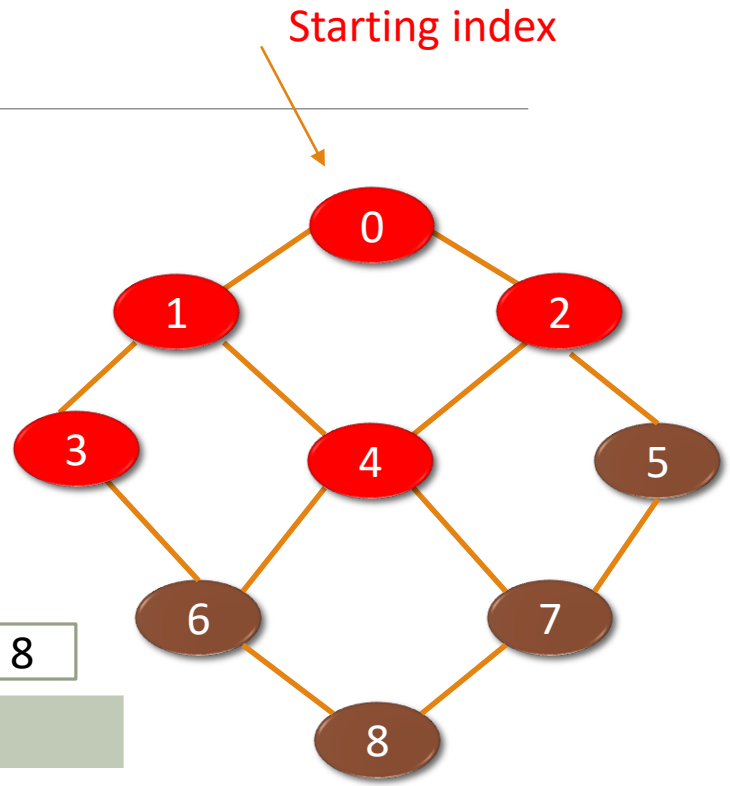
   Done

P= {1}

Adj[1] ={ 0, 3, 4}

Enqueue(3)

Enqueue(4)



Queue

0	1	2	3	4	5	6	7	8
~~0~~	~~1~~	2	3	4				

Visited array

0	1	2	3	4	5	6	7	8
1	1	1	1	1	0	0	0	0

Output

0 1 2 3 4

Breadth First Search (BFS) Algorithm

1. Enqueue starting index.
2. while Q is not empty

   Do

   P=Dequeue( )

   Enqueue all adjacent vertices of P and print

   (Check visited Array)

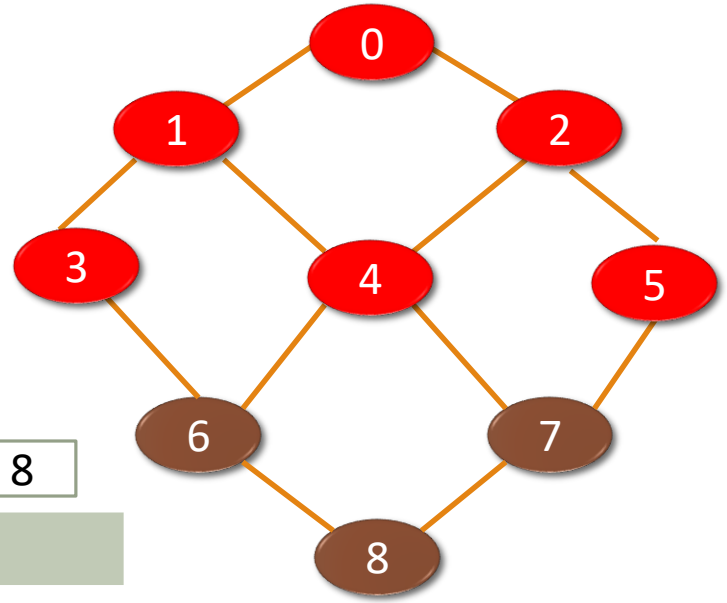
   Done

P= {2}

Adj[2] ={ 0, 4, 5}

Enqueue(5)

Starting index



Queue

0	1	2	3	4	5	6	7	8
~~0~~	~~1~~	~~2~~	3	4	5			

Visited array

0	1	2	3	4	5	6	7	8
1	1	1	1	1	1	0	0	0

Output

0 1 2 3 4 5

Breadth First Search (BFS) Algorithm

1. Enqueue starting index.
2. while Q is not empty

   Do

   P=Dequeue( )

   Enqueue all adjacent vertices of P and print

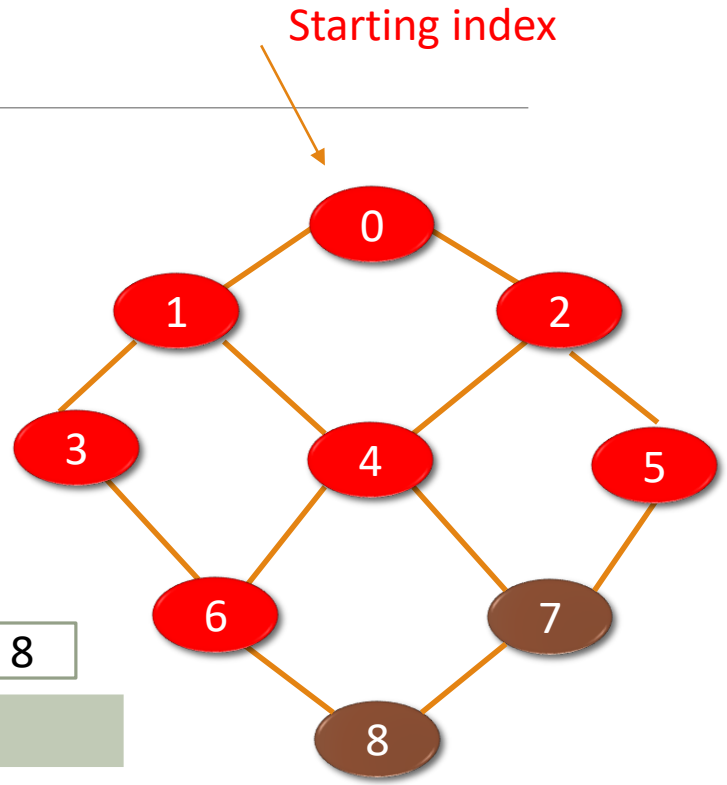
   (Check visited Array)

   Done

P= {3}

Adj[3] ={ 1, 4, 6}

Enqueue(6)



Queue

0	1	2	3	4	5	6	7	8
~~0~~	~~1~~	~~2~~	~~3~~	4	5	6		

Visited array

0	1	2	3	4	5	6	7	8
1	1	1	1	1	1	1	0	0

Output

0 1 2 3 4 5 6

Breadth First Search (BFS) Algorithm

1. Enqueue starting index.
2. while Q is not empty

   Do

   P=Dequeue( )

   Enqueue all adjacent vertices of P and print

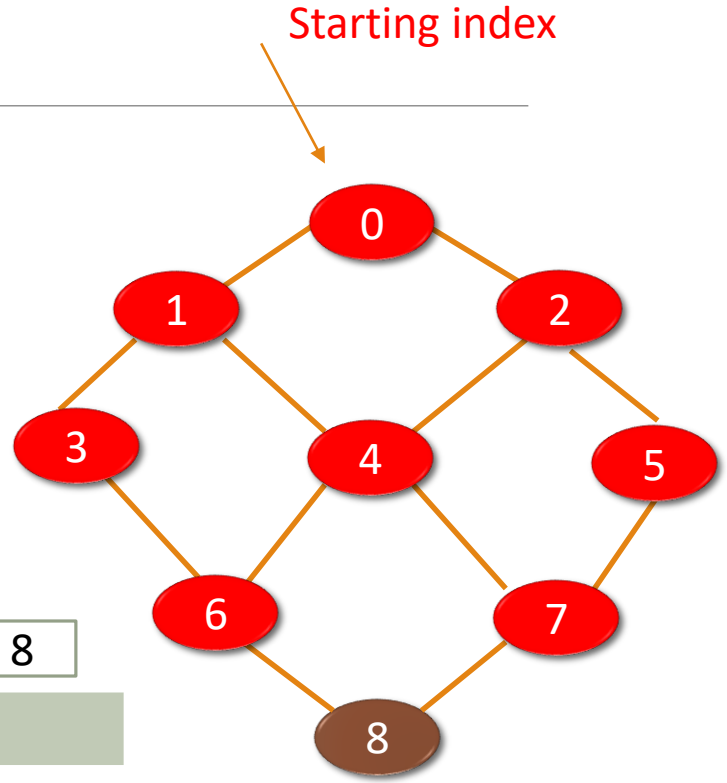
   (Check visited Array)

   Done

P= {4}

Adj[4] ={ 1, 6, 2, 7}

Enqueue(7)



Queue

0	1	2	3	4	5	6	7	8
~~0~~	~~1~~	~~2~~	~~3~~	~~4~~	5	6	7	

Visited array

0	1	2	3	4	5	6	7	8
1	1	1	1	1	1	1	1	0

Output

0 1 2 3 4 5 6 7

Breadth First Search (BFS) Algorithm

1. Enqueue starting index.
2. while Q is not empty

   Do

   P=Dequeue( )

   Enqueue all adjacent vertices of P and print

   (Check visited Array)

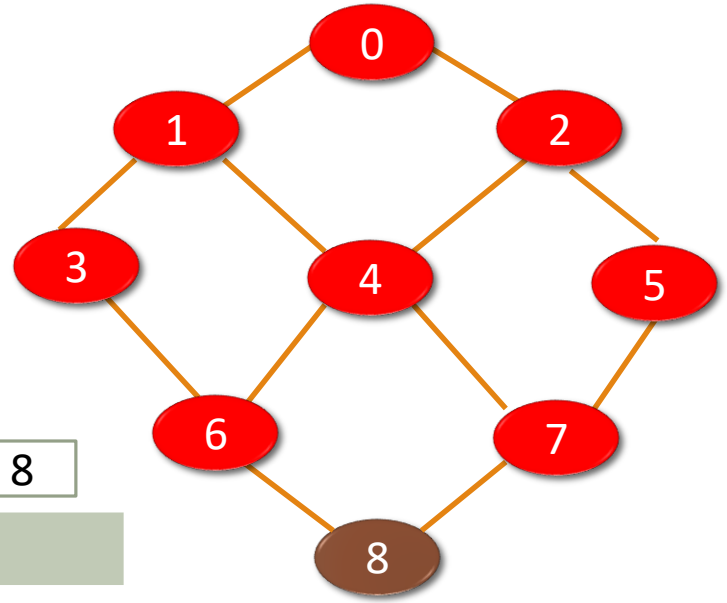
   Done

P= {5}

Adj[5] ={ 2, 4, 7}

NoEnqueue.

Starting index



Queue

0	1	2	3	4	5	6	7	8
~~0~~	~~1~~	~~2~~	~~3~~	~~4~~	~~5~~	6	7	

Visited array

0	1	2	3	4	5	6	7	8
1	1	1	1	1	1	1	1	0

Output

0 1 2 3 4 5 6 7

Breadth First Search (BFS) Algorithm

1. Enqueue starting index.
2. while Q is not empty

   Do

   P=Dequeue( )

   Enqueue all adjacent vertices of P and print

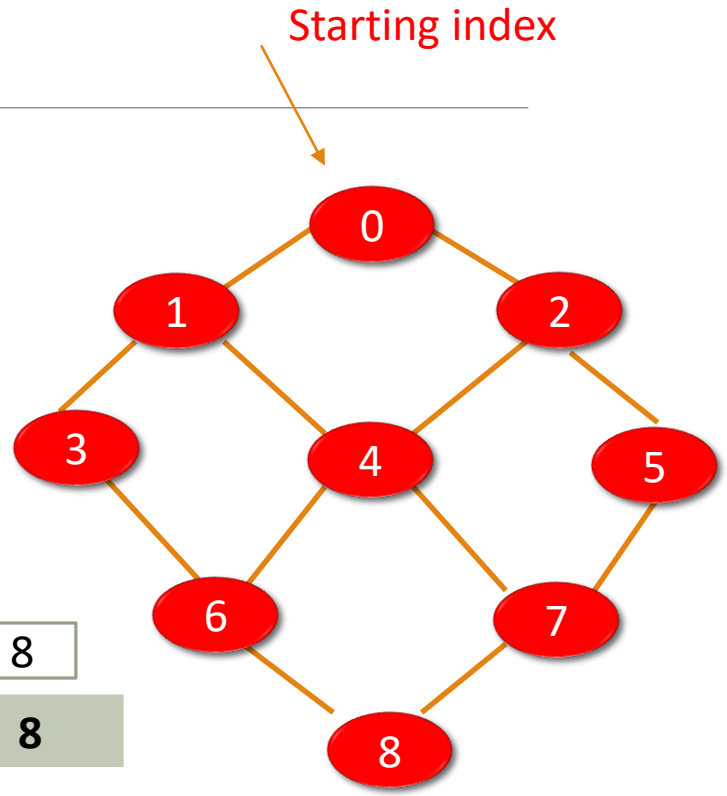
   (Check visited Array)

   Done

P= {6}

Adj[6] ={ 3, 4, 8}

Enqueue(8).



Queue

0	1	2	3	4	5	6	7	8
~~0~~	~~1~~	~~2~~	~~3~~	~~4~~	~~5~~	~~6~~	7	8

Visited array

0	1	2	3	4	5	6	7	8
1	1	1	1	1	1	1	1	1

Output

0 1 2 3 4 5 6 7 8

Breadth First Search (BFS) Algorithm

1. Enqueue starting index.
2. while Q is not empty

   Do

   P=Dequeue( )

   Enqueue all adjacent vertices of P and print

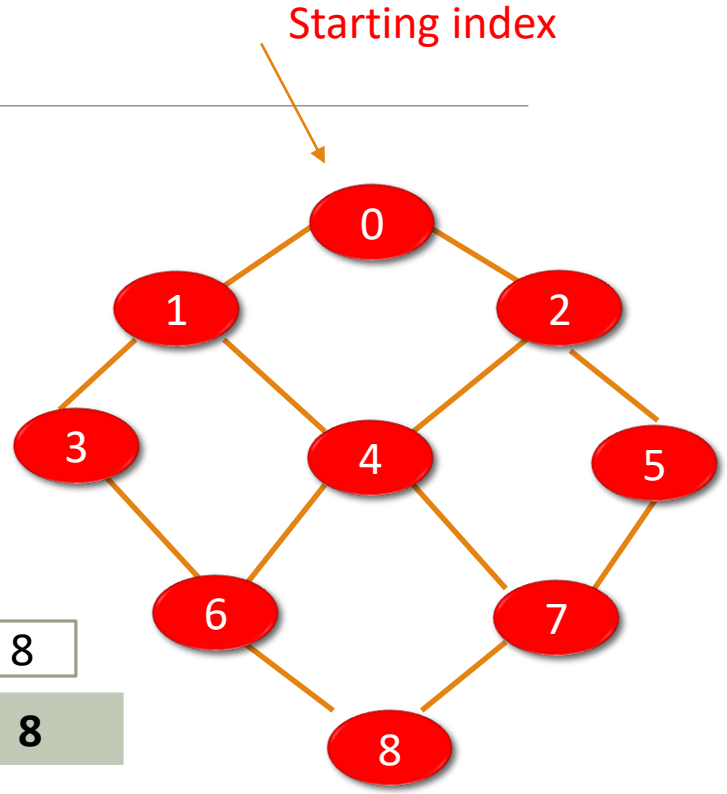
   (Check visited Array)

   Done

P= {7}

Adj[7] ={ 4, 5, 8}

NoEnqueue.



Queue

0	1	2	3	4	5	6	7	8
~~0~~	~~1~~	~~2~~	~~3~~	~~4~~	~~5~~	~~6~~	~~7~~	8

Visited array

0	1	2	3	4	5	6	7	8
1	1	1	1	1	1	1	1	1

Output

0 1 2 3 4 5 6 7 8

Breadth First Search (BFS) Algorithm

1. Enqueue starting index.
2. while Q is not empty

   Do

   P=Dequeue( )

   Enqueue all adjacent vertices of P and print

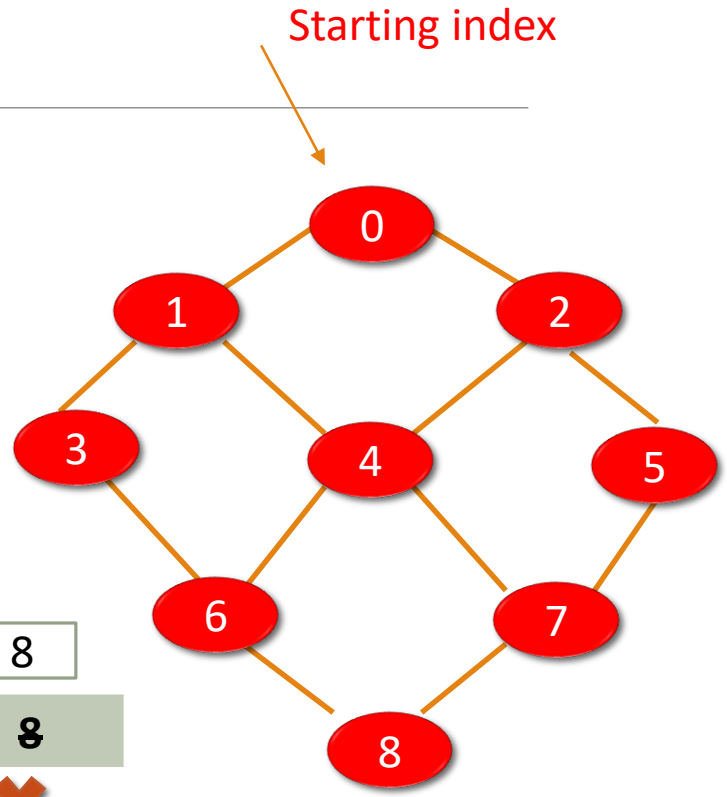
   (Check visited Array)

   Done

P= {8}

Adj[8] ={ 6, 7}

NoEnqueue.



Queue

0	1	2	3	4	5	6	7	8
~~0~~	~~1~~	~~2~~	~~3~~	~~4~~	~~5~~	~~6~~	~~7~~	~~8~~

Visited array

0	1	2	3	4	5	6	7	8
1	1	1	1	1	1	1	1	1

Output

0 1 2 3 4 5 6 7 8

Breadth First Search (BFS) Algorithm

1. Enqueue starting index.
2. while Q is not empty

   Do

   P=Dequeue( )

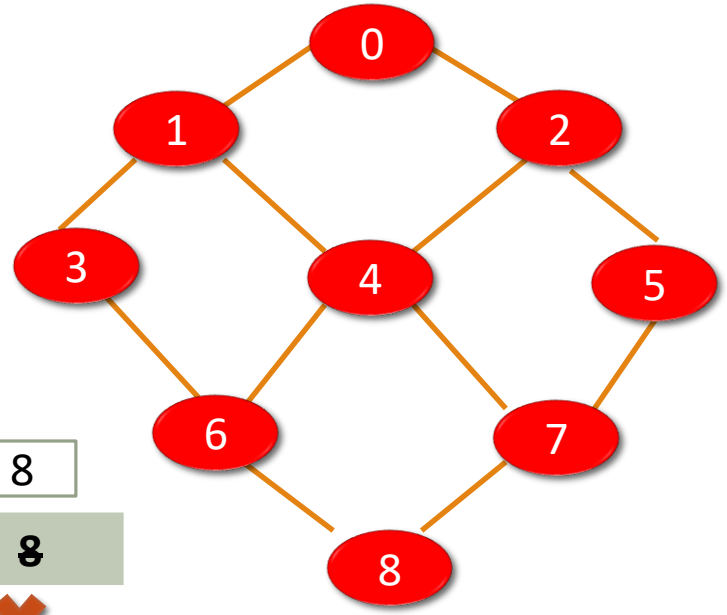
   Enqueue all adjacent vertices of P and print

   (Check visited Array)

   Done

Queue is Empty. Stop the while loop.

Starting index



Queue

0	1	2	3	4	5	6	7	8
~~0~~	~~1~~	~~2~~	~~3~~	~~4~~	~~5~~	~~6~~	~~7~~	~~8~~

Visited array

0	1	2	3	4	5	6	7	8
1	1	1	1	1	1	1	1	1

Output

0 1 2 3 4 5 6 7 8

Depth First Search (DFS) Algorithm

1. Push starting index.
2. while Stack is not empty

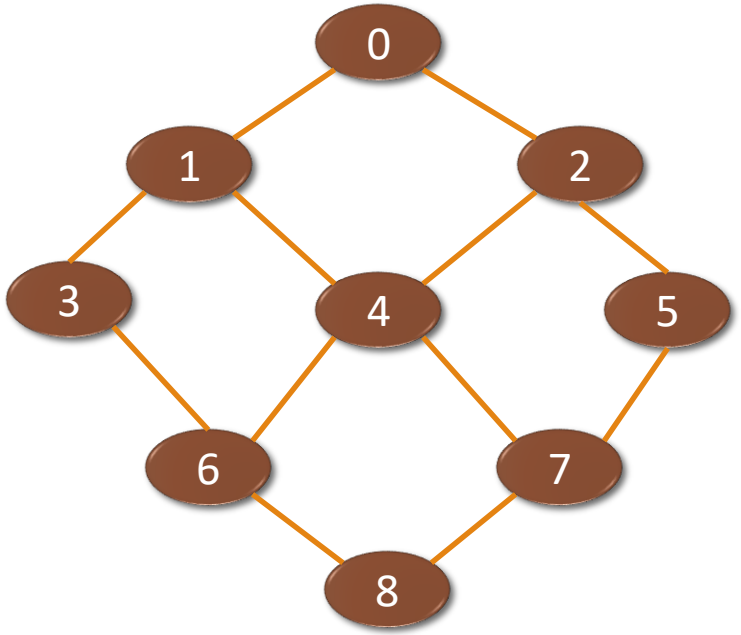
   Do

      P=POP( )

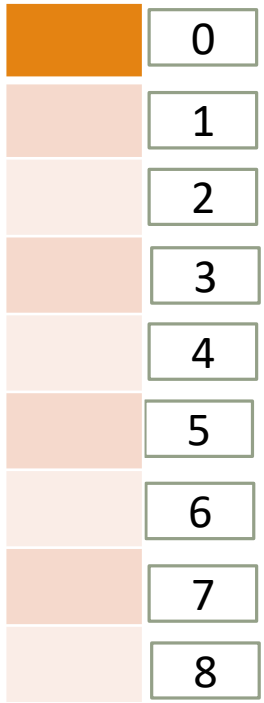
      Push one adjacent node and print.

      [If no valid adjacent vertex for P, then POP( )]

   Done



Stack



Visited array

0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0

Valid: unvisited adjacent vertex

Depth First Search (DFS) Algorithm

1. Push starting index.
2. while Stack is not empty

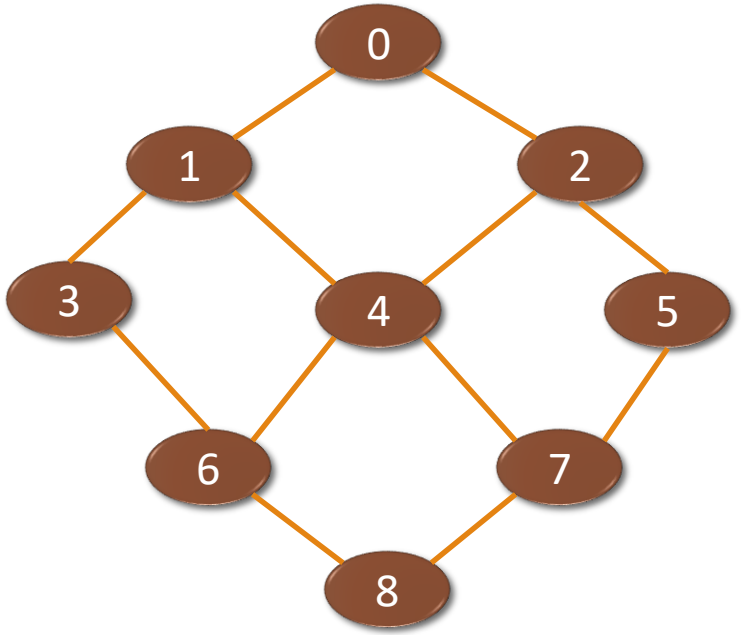
   Do

      P=POP( )

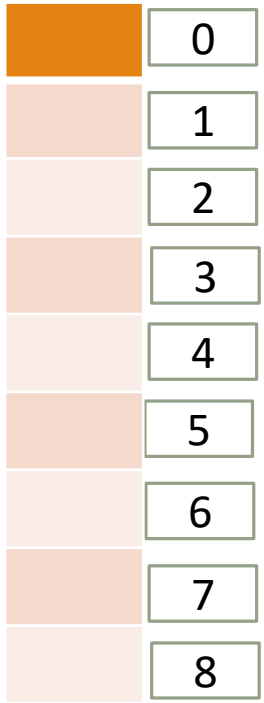
      Push one adjacent node and print.

      [If no valid adjacent vertex for P, then POP( )]

   Done



Stack



Trace the remaining steps . .. .

Visited array

0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0

Valid: unvisited adjacent vertex

END